

Name - Saikat Sheet
Batch - 25SUB5936_Linux System Programming
ID - 68500

1. Explain the difference between proprietary and open-source software with examples.

Ans> The primary difference between proprietary and open-source software lies in **ownership, access to source code, and licensing.**

1. Open-Source Software (OSS)

Open-source software is characterized by its transparent nature. The **source code**—the actual instructions written by programmers—is freely available for anyone to inspect, modify, and distribute.

- **Development:** Usually driven by a community of volunteers and organizations.
- **Cost:** Often free to use (though support and enterprise features may cost money).
- **Flexibility:** Users can fix bugs themselves or add custom features without waiting for an official update.
- **Transparency:** Because the code is public, it is easier to audit for security vulnerabilities or "backdoors."

Examples:

- **Operating Systems:** Linux (Ubuntu, Fedora, Debian).
- **Web Browsers:** Firefox, Chromium.
- **Programming:** Python, C++, and the GNU Compiler Collection (GCC).
- **Office Suites:** LibreOffice.

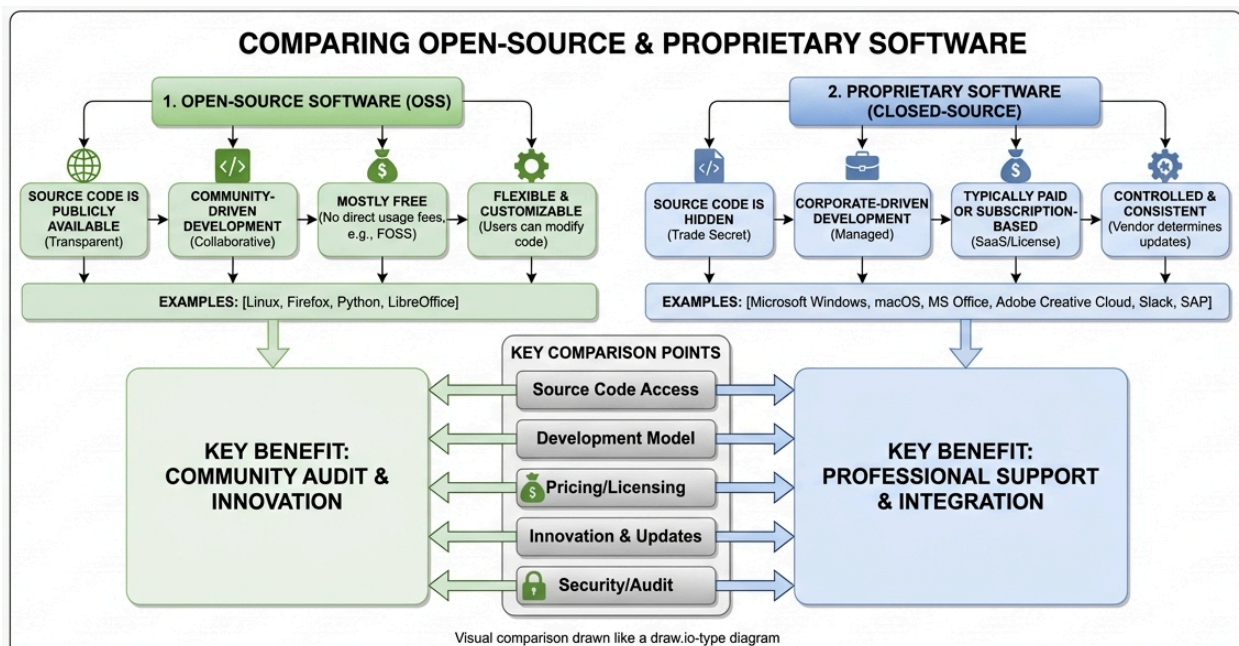
2. Proprietary Software

Proprietary software (also known as "closed-source") is owned by an individual or a company. The source code is a closely guarded trade secret.

- **Development:** Managed and developed exclusively by the owner.
- **Cost:** Usually involves a purchase price, subscription fee (SaaS), or licensing costs.
- **Control:** The vendor decides when to release updates, which features to add, and when to stop supporting the product.
- **Support:** Typically comes with professional technical support and a warranty, which is often preferred by large corporations.

Examples:

- **Operating Systems:** Microsoft Windows, macOS.
- **Productivity:** Microsoft Office 365, Adobe Creative Cloud (Photoshop).
- **Communication:** Slack, Discord.
- **Enterprise:** SAP, Oracle Database.



2. Why is MVC considered beneficial in large-scale applications?

Ans>

1. Enhanced Maintainability and Scalability

Large applications are never "finished"; they are constantly updated. By isolating the data logic (Model) from the user interface (View), developers can make changes to one without breaking the other.

- **Independent Updates:** You can redesign the entire website UI (View) without touching the database logic or the API routes (Model).
- **Component Reuse:** A single Model can provide data for multiple Views. For instance, the same underlying data can be displayed on a desktop web browser, a mobile app, and a PDF report simultaneously.

2. Simultaneous Development

In a large-scale project, you often have different teams working together. MVC allows these teams to work in parallel without stepping on each other's toes:

- **Front-end teams** focus on the **View**, styling elements and refining the user experience.
- **Back-end teams** focus on the **Controller** and **Model**, handling data processing and server-side logic.
- This specialization speeds up the development lifecycle and reduces the risk of merge conflicts in version control.

3. Ease of Testing

Testing a massive, monolithic application is a nightmare. MVC makes it significantly easier by allowing for **Unit Testing**:

- Because the **Model** is independent of the UI, you can write automated tests to verify that data is being saved or calculated correctly without needing to launch a browser or simulate user clicks.

- The **Controller** can be tested to ensure it handles requests and inputs properly before sending them to the Model.
-

3. Compare SOA and Microservices with real-world analogies.

Ans>

When comparing **Service-Oriented Architecture (SOA)** and **Microservices**, it helps to think about how a large organization manages its different functions.

1. The Real-World Analogies

SOA: The "Government Department" Model

Imagine a large city government. You have different departments: the Department of Motor Vehicles (DMV), the Tax Office, and the City Planning Office.

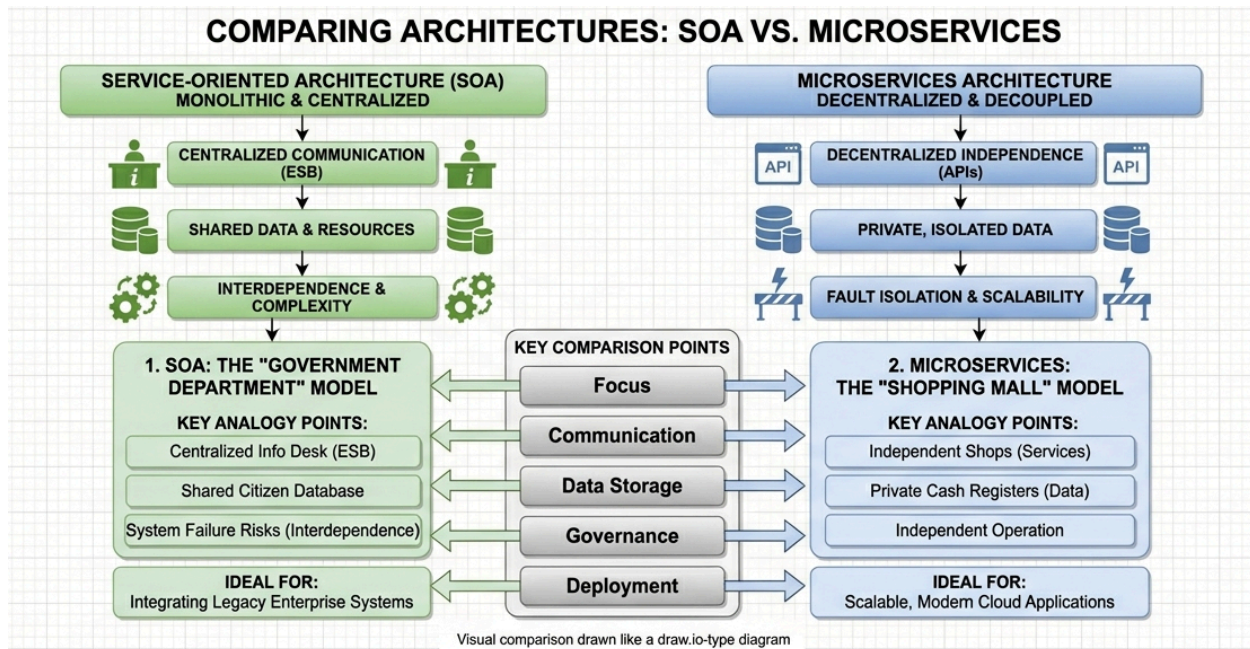
- **Centralized Communication:** If you want to start a business, you might go to a central "information desk" (the **Enterprise Service Bus** or **ESB**) that tells you which forms to take to which department.
- **Shared Resources:** All these departments likely share the same central database of citizens and the same building infrastructure.
- **Interdependence:** If the central database goes down, every department stops working.

Microservices: The "Shopping Mall" Model

Imagine a shopping mall filled with independent stores: a coffee shop, a clothing store, and a cinema.

- **Decentralized Independence:** Each store is its own business. The coffee shop has its own staff, its own cash register, and its own inventory system. They don't need a central manager to function.
- **Specific Communication:** If the coffee shop needs to verify a mall gift card, it talks directly to the mall's card service via a simple, standard method (like a card reader/API).

- **Isolation:** If the clothing store has a power outage, you can still buy a latte at the coffee shop. The failure of one "service" doesn't shut down the others.



4. Why is REST risky in microservices ?

Ans> It introduces specific "risks" and challenges that can lead to system-wide failures if not managed properly.

1. Tight Coupling and Cascading Failures

The biggest risk with REST is its **synchronous** nature. When Service A calls Service B via REST, it must wait for a response.

- **The Chain Reaction:** If Service B is slow or down, Service A's threads become blocked while waiting. If Service C also calls Service A, the entire chain can "hang."
- **Dependency:** This creates a temporal coupling where the availability of one service is strictly dependent on the availability of another.

2. The "Chatty" API Problem

Microservices often need data from multiple sources. In a REST-based system, this leads to significant overhead:

- **Multiple Round Trips:** A single user request might require a frontend to make five different REST calls to five different services, leading to high latency.
- **Over-fetching:** REST endpoints often return a fixed data structure. A service might only need a user's ID, but the REST response sends the entire user profile, wasting bandwidth.

3. Lack of Type Safety

REST typically relies on JSON, which is just a string of text.

- **Contract Breaking:** If the "Order Service" changes a field name from `price` to `cost`, it won't know it has broken the "Invoice Service" until the code actually runs and fails.
- **Manual Mapping:** Developers must manually ensure that both the sender and receiver are using the exact same data format, which is prone to human error in large-scale systems.

4. Complex Service Discovery

In a microservices environment, instances of services are constantly spinning up and down with different IP addresses.

- **The Risk:** Relying on static URLs for REST calls is impossible. You must implement a complex "Service Discovery" layer (like Eureka or Consul) to help services find each other, adding another point of failure to the system.
-

5. What is service discovery?

Ans> In a microservices architecture, **Service Discovery** is the mechanism that allows services to find and communicate with each other over a network. Since microservice instances are dynamic—scaling up, scaling down, or failing and being replaced—their IP addresses and ports change constantly.

The Core Components

To make this work, the architecture typically relies on two main pieces:

- **Service Registry:** A central database containing the network locations of all active service instances.
- **Service Check-in/Discovery:** The process where a service "registers" its location when it starts up and "queries" the registry when it needs to find another service.

How It Works: Two Main Patterns

There are two primary ways to implement this "automated phone book" lookup:

1. Client-Side Discovery

The **service consumer** is responsible for looking up the address.

- The client queries the **Service Registry**.
- The client then uses a load-balancing algorithm to choose one of the available service instances.
- **Example:** Netflix Eureka.

2. Server-Side Discovery

The **service consumer** sends a request to a **Load Balancer**.

- The client doesn't know about the registry. It just hits a fixed endpoint (the load balancer).
- The load balancer queries the **Service Registry** and routes the request to an available instance.

- **Example:** AWS Elastic Load Balancing (ELB) or Kubernetes Services.
-

6. Why are events better than REST in large systems ?

Ans>

In large-scale systems, moving from a request-driven model (REST) to an event-driven model is often the key to maintaining stability and speed. While REST is like a phone call that requires both people to be on the line, an event-driven architecture is like a high-speed notification system.

Here is why events are often superior for complex environments:

1. Decoupling and "Fire and Forget"

In a REST-based system, Service A must know that Service B exists, know its URL, and wait for it to finish. In an event-driven system, Service A simply broadcasts an event (e.g., "Order Placed") to a message broker.

- **Producer Independence:** The service creating the event doesn't care who consumes it or what they do with it.
- **Reduced Latency:** Because the producer doesn't wait for a response, the user gets an immediate "Success" message while the heavy lifting happens in the background.

2. Elimination of Cascading Failures

As we discussed with REST risks, if one service in a chain fails, the whole chain often hangs.

- **Buffering:** If a downstream service (like the "Email Service") goes offline, the events are simply stored in the message broker (like Kafka or RabbitMQ).

- **Resilience:** Once the service comes back online, it picks up exactly where it left off. The "Order Service" was never slowed down or crashed by the "Email Service" being down.
-

7. What challenges do microservices introduce?

Ans> Transitioning to microservices solves many problems found in monolithic systems, but it essentially trades one set of problems for another.

1. Operational Overhead

Managing a single application is relatively simple. Managing 50 or 100 individual services is a massive undertaking.

- **Infrastructure Management:** Each service needs its own CI/CD pipeline, monitoring, logging, and deployment strategy.
- **Resource Usage:** Running multiple services, each with its own runtime (like a Java VM or a Python interpreter) and its own database, consumes significantly more memory and CPU than a single monolith.

2. Network Latency and Reliability

In a monolith, components talk to each other via fast, reliable in-memory calls. In microservices, they talk over the network.

- **The "Fallible" Network:** Networks can be slow or drop packets. You must write complex code to handle timeouts, retries, and "circuit breaking" (stopping calls to a service that is clearly struggling).
- **Performance Hits:** Every network hop adds milliseconds to a request. If a user's action triggers a chain of five service calls, that latency adds up quickly.

3. Security Challenges

A monolith has one "front door" to defend. Microservices have a much larger "attack surface."

- **Internal Security:** You must ensure that Service A is actually authorized to talk to Service B. This often requires implementing Mutual TLS (mTLS) or complex JWT (JSON Web Token) validation between every single service.
 - **Data in Transit:** Sensitive data is constantly moving across the internal network, requiring encryption at every step.
-

8. How do microservices and EDA complement each other ?

Ans> If Microservices are the "actors" in a play, EDA is the "script" that tells them when to enter the stage without everyone having to talk at once.

1. Decentralization of Logic

Microservices aim to be independent. REST (request-response) often forces them to wait for each other, creating a "distributed monolith." EDA fixes this.

- **The Complement:** Instead of Service A telling Service B to do something, Service A simply shouts into the room, "I have finished my task!" (the Event).
- **Result:** Any service that cares about that information can react on its own time. This keeps services truly decentralized.

2. Data Consistency (Saga Pattern)

Since every microservice has its own database, you can't use a traditional database "transaction" to update two things at once.

- **The Complement:** EDA uses events to maintain **Eventual Consistency**.
 - Service A updates its database and sends an event.
 - Service B hears the event and updates its database.

- **Result:** Even though the updates don't happen at the exact same microsecond, the system eventually reaches the correct state without locking up resources.

3. Simplified Extensibility

One of the main goals of microservices is to allow teams to add new features quickly.

- **The Complement:** Because services only interact with the **Event Stream**, you can add a brand-new service (like a "Real-time Analytics Service") that listens to existing events without changing a single line of code in the old services.
- **Result:** You get a "Plug-and-Play" architecture.

